ClaimGuard

Real-time AI risk scoring for insurance claims on IBM Z

Pay honest claims in hours. Stop fraudulent ones before the money leaves.

Event	IBM Z Datathon 2026
Theme	Real-time AI on Transactional Data (Theme 1)
Live demo	https://change-illustrations-tooth-complete.trycloudflare.com (running on IBM LinuxONE, s390x)

99%	**37%**	**0.91**	**Live**
fraud value stopped before payout	claims paid instantly	ROC-AUC (time hold-out)	on IBM LinuxONE (s390x)

1. Executive summary

Insurers check most claims manually, after they are filed and often after they are paid. Honest customers wait days or weeks, fraud still slips through, and losses return to everyone as higher premiums. **ClaimGuard scores every claim the instant it is filed**, inside the claims transaction, and routes it to one of three lanes: fast-track payout, a one-document check, or an investigator. Every decision carries exact reason codes, and IBM Granite turns them into a short note for investigators.

On a strict time-based hold-out of the 12,000 most recent claims, ClaimGuard held **99%** of fraudulent claim value before payout, paid **37%** of claims with no manual work, and caught **81%** of fraud versus **62%** for red-flag rules at the same alert volume. The system is **trained and served on a real IBM LinuxONE server** today, with a public demo.

2. The problem

Two customers show why timing matters:

- **Priya** has a minor accident and claims Rs 18,000. She is honest but waits about two weeks, because every claim goes through the same manual queue.
- **Rahul** claims Rs 2.4 lakh for a 'total loss' on a policy bought 9 days earlier, with no police report, through a garage used by seven other customers who share his phone number.

Manual checks treat them the same. Rule-based red flags catch only obvious cases, raise many false alarms, and cannot see hidden links between customers, providers and agents, which is how organised rings operate.

3. The solution

Lane	What happens	Share of test claims
Fast-track	Paid straight through, no manual work	37% (only 3% of fraud enters this lane)
Document check	Customer uploads one extra document in-app	59%
Investigate	Payout held; investigator gets an explained case	4% (77% hit-rate)

No claim is ever rejected by a machine: a human decides every high-risk case. Thresholds are business policy (at most 3% of fraud may be fast-tracked, and investigators should be right about 80% of the time), not hidden inside the model.

4. Architecture and how it works

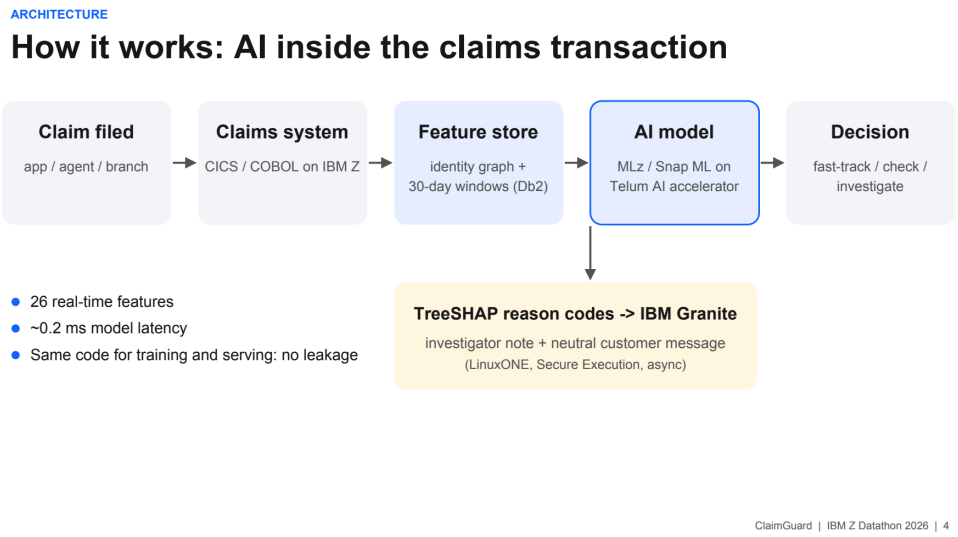


Figure 1. ClaimGuard scores claims inside the claims transaction.

- **Stream.** Each claim event arrives from the core claims system (replayed feed in the prototype).
- **Real-time features (26).** Amount vs actuarial tariff, share of sum insured, policy age, reporting delay, incident hour, police report / pre-authorisation, documents and channel. Plus an **identity graph** (union-find over shared phones and bank accounts) and **rolling 30-day windows** per garage/hospital and agent (claim spikes, billing level, late-night share, linked-group share).
- **Predict.** Gradient-boosted trees (XGBoost) give a risk score in about a millisecond.
- **Decide.** A policy engine maps the score to a lane.
- **Explain.** Exact TreeSHAP reason codes for every decision. IBM Granite (watsonx.ai or local) rewrites them into an investigator note in English, Hindi, Tamil, Bengali or Marathi, and never changes the score.

The same *build_features* → *observe* code path is used for training (replaying history in time order) and for live scoring, so there is no train/serve skew and no future leakage.

5. Data methodology

Real claims data is confidential, so ClaimGuard includes a simulator of an Indian insurer's motor and health claims: 60,000 claims, 42,000 customers, 680 garages and hospitals, 300 agents, one year. Fraud mechanisms follow publicly documented red flags (IRDAI, Insurance Information Bureau of India, NICB):

- opportunistic **inflation** of genuine bills
- **early-policy** claims
- **phantom** claims with weak paperwork
- **provider billing** fraud by colluding garages and hospitals
- organised **rings** sharing phones and bank accounts, staging late-night incidents through one provider and agent

To keep the task honest, genuine and fraudulent distributions overlap (honest new policies, late reports, families sharing a phone), and 0.5% label noise mimics imperfect investigation outcomes. Evaluation uses a strict **time-based split**: 70% train, 10% validation (for thresholds) and 20% test, never a random shuffle.

6. Results

Evaluated on the 12,000 most recent claims (fraud rate 7%). Because fraud is rare, accuracy is misleading; we report ranking quality (ROC-AUC, PR-AUC) and business outcomes.

Metric	Value	Meaning
ROC-AUC	0.907	How well risky claims rank above safe ones (0.5 = random)
PR-AUC	0.659	About 10× better than random (0.067)
Fraud value stopped before payout	99%	Rs 14.78 Cr of Rs 14.91 Cr
Claims fast-tracked	37%	Only 3% of fraud enters this lane
Investigator hit-rate	77%	Queue is 4% of claims
Fraud caught vs rules (same alerts)	81% vs 62%	2,336 alerts; precision 28% vs 21%
Scoring latency	~0.2–1 ms	Per claim, on a laptop and on 3 LinuxONE vCPUs

Fraud stopped before payout, by scheme

Scheme	early policy	inflation	phantom	provider billing	staged ring
Not fast-tracked	100%	97%	99%	97%	100%

What the model relies on most

Feature	Gain
Linked-customer network	172.9
Provider linked-group share	141.8
Amount vs typical cost	60.5
Reporting delay	33.7
Policy age	27.8
Documents complete	26.4
Share of sum insured	19.4
Agent linked-group share	12.3

Graph and provider features rank highest, which confirms that linking customers, providers and agents in real time adds signal that per-claim rules cannot capture.

7. The prototype

The dashboard is live at https://change-illustrations-tooth-complete.trycloudflare.com, served from IBM LinuxONE.

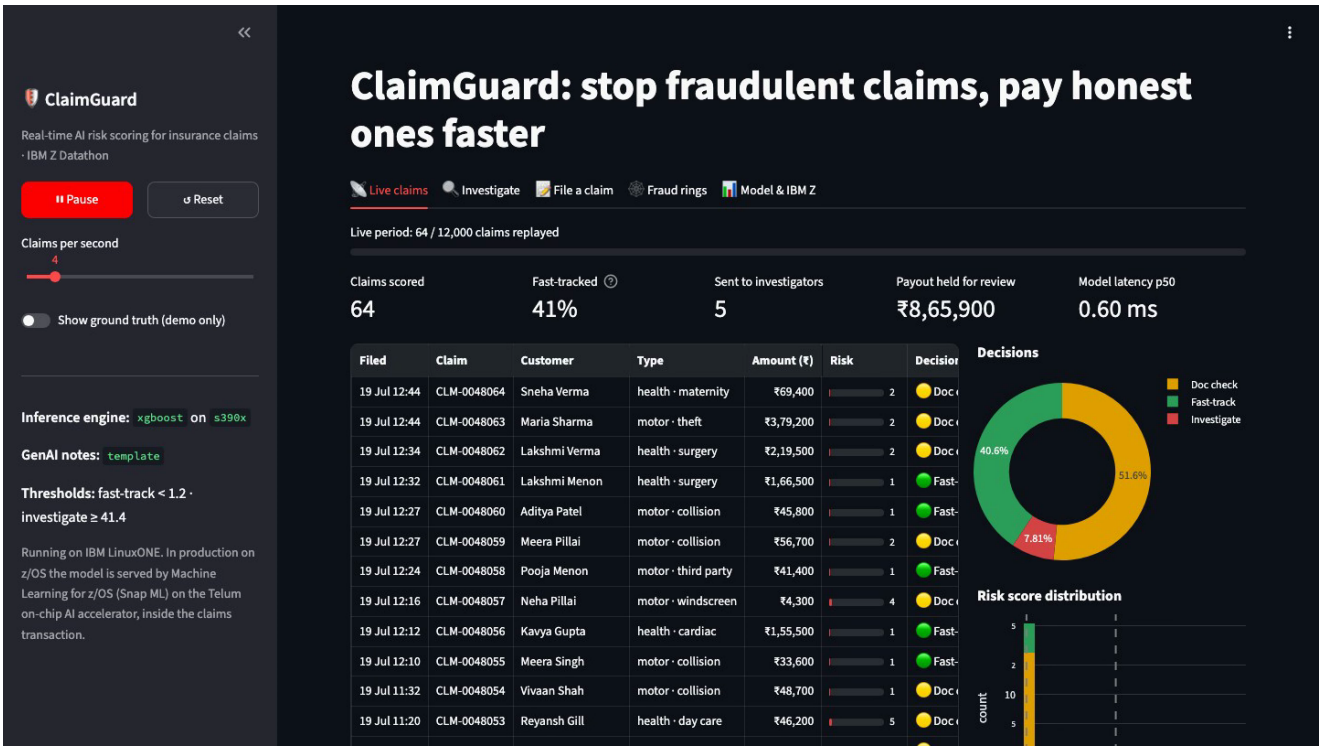


Figure 2. Live claims: every claim scored in about a millisecond and routed to a lane. The sidebar shows the s390x (IBM Z) architecture.



Figure 3. Investigator view: risk gauge, exact SHAP reason codes and contribution chart.

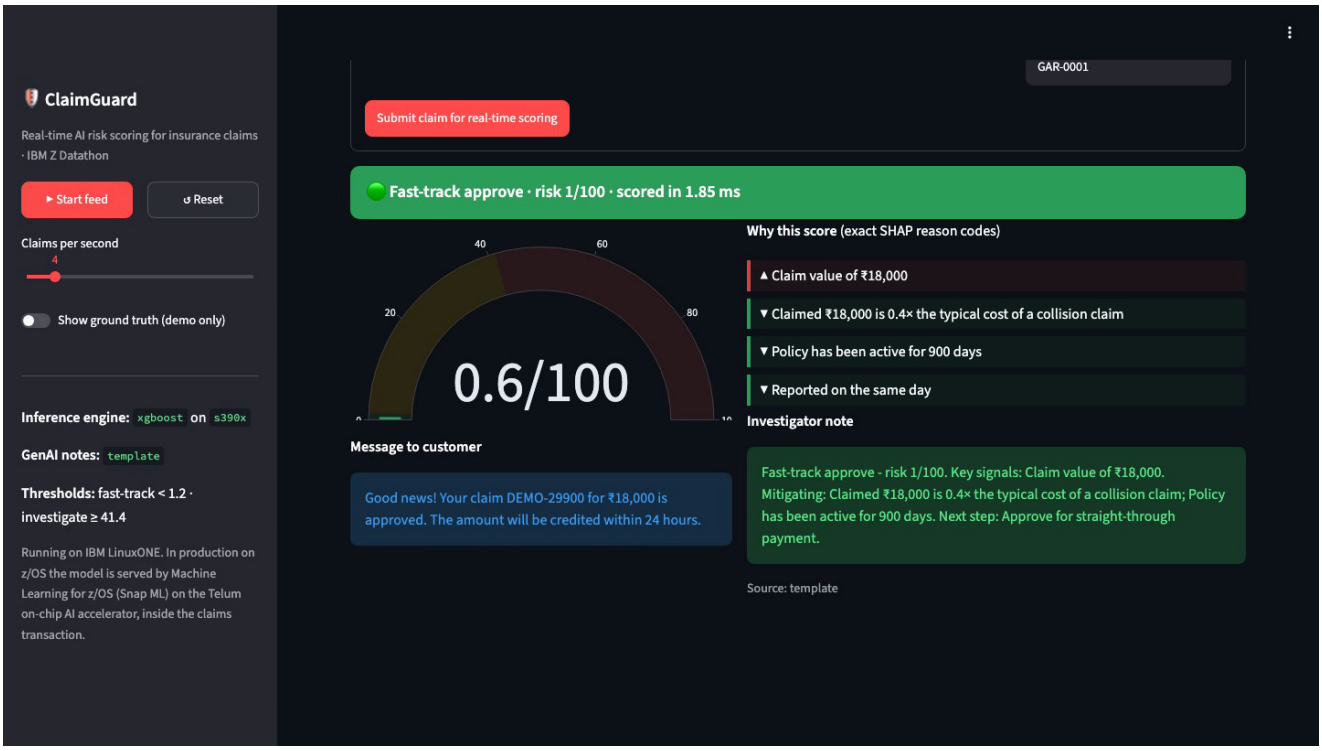


Figure 4. Priya's genuine claim: risk 0.6/100, fast-tracked, positive customer message.

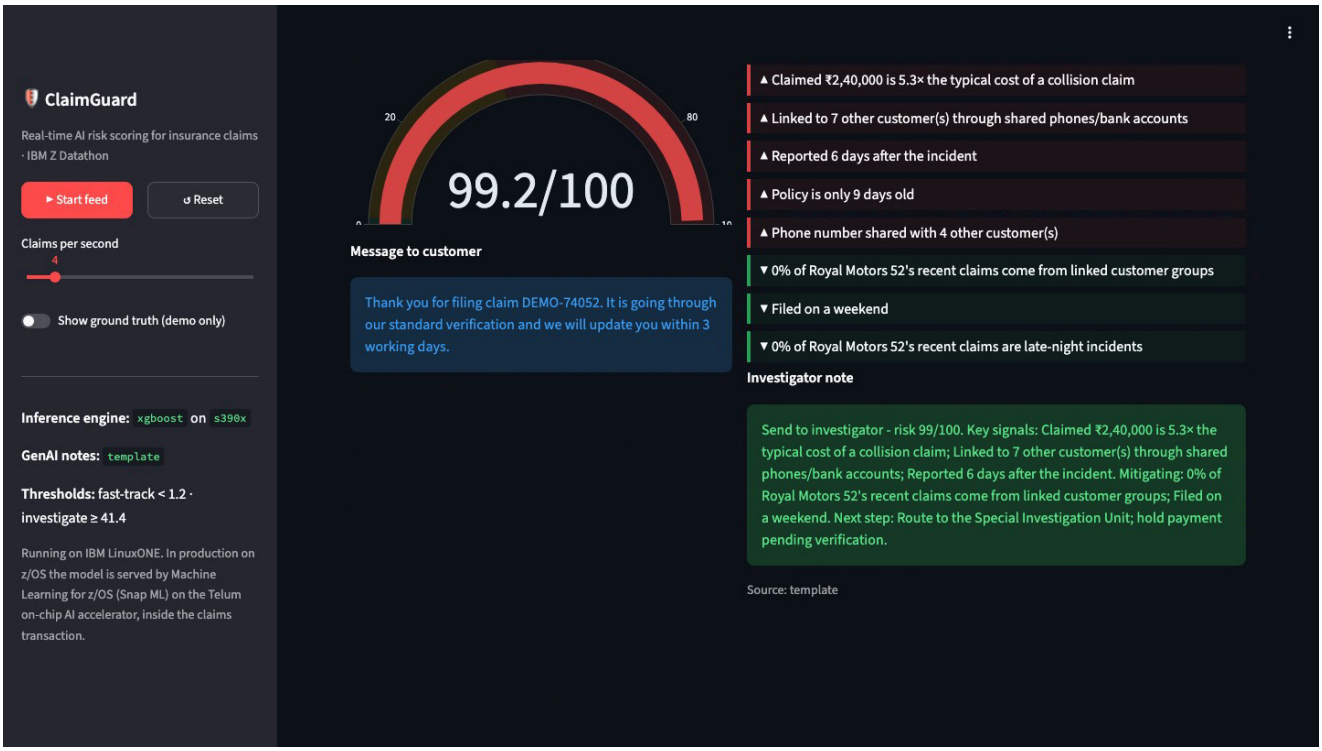


Figure 5. Rahul's claim: risk 99/100, sent to an investigator, with a neutral customer message.

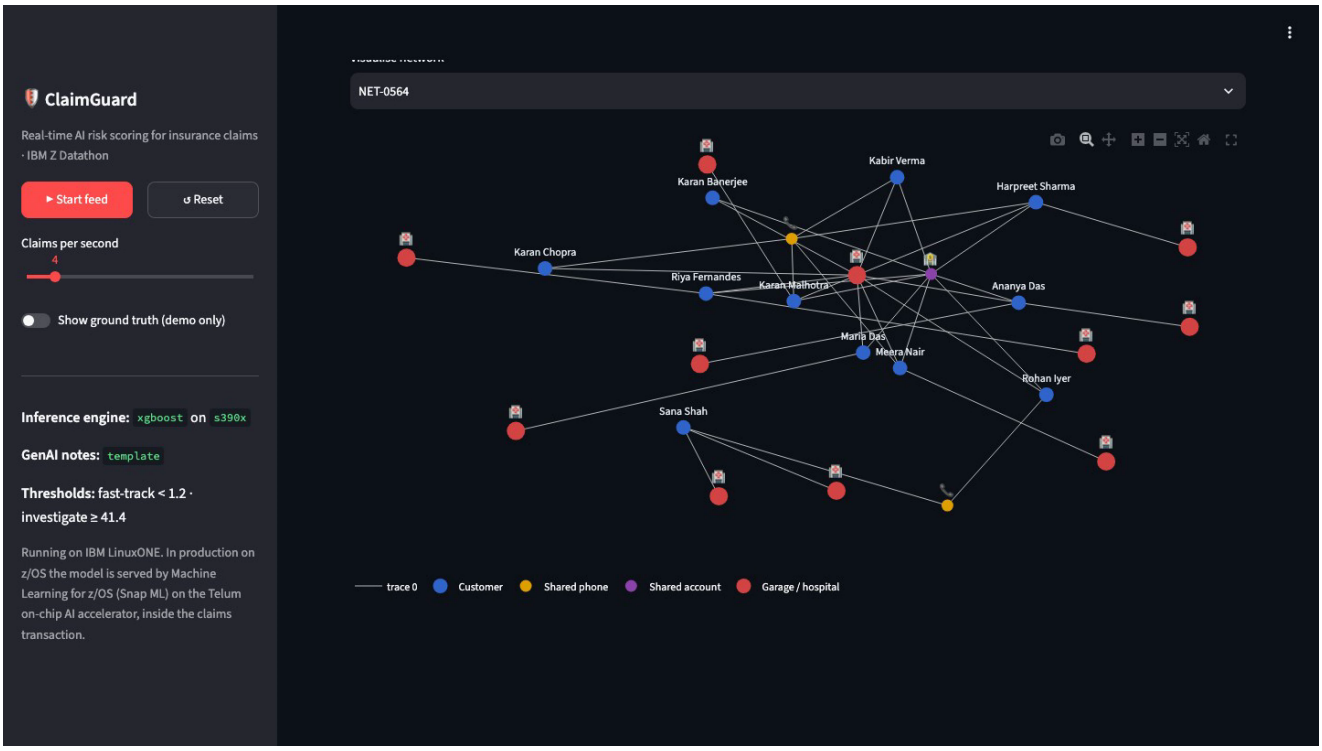


Figure 6. Identity graph: linked customers funnelling claims into one garage.

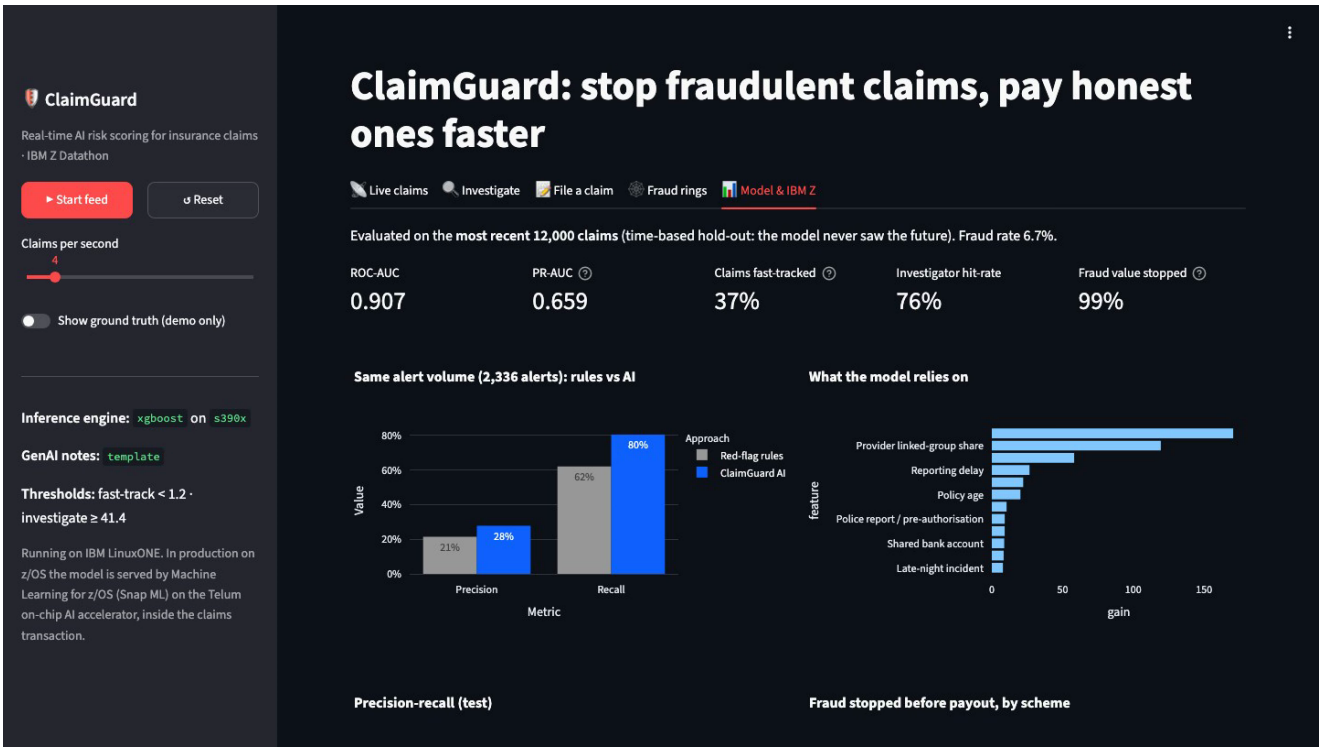


Figure 7. Model and IBM Z tab: metrics, rules vs AI, and the deployment map.

8. IBM Z and LinuxONE

Running live on IBM LinuxONE

ClaimGuard is trained, tested and served on a real IBM LinuxONE virtual server (s390x, RHEL 9.8). The dashboard, the scoring API and nginx run as systemd services, and a Cloudflare tunnel compiled for s390x gives the public link.

Check on s390x	Result
Training (60k claims, 26 features)	25 s, ROC-AUC 0.907, 99% of fraud value stopped
Smoke tests	6 / 6 pass
Live scoring latency	~0.3–1 ms per claim (3 vCPUs)
Python AI stack	numpy, pandas, scikit-learn, XGBoost, Apache Arrow and pyarrow compiled from source for s390x
Big-endian fixes	Tables rendered as HTML and Plotly arrays sent as JSON lists, because browser libraries assume little-endian data

Production path on IBM Z

Capability	Role in ClaimGuard
Telum on-chip AI accelerator	Scores 100% of claims synchronously inside the transaction, with no network hop
Machine Learning for z/OS + IBM Snap ML	Serves the tree model in-transaction to CICS/COBOL claims apps (Snap ML tree inference uses the Telum accelerator)
Db2 for z/OS	Feature store next to the claim records, so data never leaves the platform
Pervasive encryption + Crypto Express	Health and financial data encrypted at rest and in flight, with HSM-protected keys
Secure Execution / Hyper Protect	Granite and the dashboard run in confidential containers on LinuxONE
IBM Z Deep Learning Compiler	Path for future neural-network variants exported as ONNX

The z/OS / Machine Learning for z/OS integration is the designed production path and was not tested in this prototype; the LinuxONE deployment is live.

9. Responsible AI

- **Human in the loop:** high risk means a human decides; the model never rejects a claim.
- **Explainable:** exact TreeSHAP reason codes for every decision.
- **Non-accusatory:** customers receive neutral, helpful messages; GenAI only rephrases facts.
- **Privacy by design:** the model uses phone and bank *linkage*, not raw identities; on IBM Z data stays encrypted on-platform.
- **Fairness:** no protected attributes (gender, religion, caste); age and tenure to be monitored for disparate impact before production.

10. Limitations and next steps

- Results come from synthetic data; a pilot on an insurer's historical claims is the next step (the pipeline is schema-driven).
- 59% of claims get a one-document check under the strict 3% fraud-leak policy; insurers can relax it to fast-track more.
- Granite notes use a deterministic template unless watsonx.ai or a local Granite model is configured.
- The free public link changes if the tunnel restarts; a named Cloudflare tunnel or domain would make it permanent.
- Next: Machine Learning for z/OS / Snap ML in a CICS claims flow, drift monitoring and retraining, then crop, life and travel claims.

11. Impact

Who	Benefit
Policyholders	Honest claims paid in hours instead of weeks
Insurers	Fraud stopped before payout; lower handling cost
Investigators	A short, explained queue with fraud-ring graphs
Regulators	Transparent, explainable, human-reviewed decisions
Society	Lower premiums, more trust in insurance, wider coverage

12. Technology and open-source resources

Layer	Tools
Data and ML	Python, NumPy, pandas, scikit-learn, XGBoost, ONNX, onnxmltools, ONNX Runtime
Graph	Union-find (custom), NetworkX
Serving	FastAPI, Uvicorn, nginx, systemd, Cloudflare tunnel (cloudflared)
Dashboard	Streamlit, Plotly, Apache Arrow
GenAI	IBM Granite (watsonx.ai or Ollama)
IBM Z	IBM LinuxONE (s390x), RHEL 9; production path MLz / Snap ML, Telum, Db2 for z/OS

Links: XGBoost github.com/dmlc/xgboost · scikit-learn scikit-learn.org · ONNX onnx.ai · ONNX Runtime onnxruntime.ai · FastAPI fastapi.tiangolo.com · Streamlit streamlit.io · Plotly plotly.com/python · NetworkX networkx.org · IBM Granite huggingface.co/ibm-granite · IBM zDLC github.com/IBM/zDLC · cloudflared github.com/cloudflare/cloudflared